

Version Control & Advanced Git

Category: **Source Control & Collaboration Engineering** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Version Control & Advanced Git**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

- Directed Acyclic Graph (DAG) & Object Model:** Git stores data as content-addressable objects in `.git/objects` identified by SHA-1/SHA-256 hashes: Blobs (file contents), Trees (directory structures), Commits (pointers to trees and parent commits), and Annotated Tags.
Production Application: Understanding the DAG prevents fear of rebasing or resetting; as long as an object was committed, it exists in the object store and can be restored using the reflog.
- The Three Trees: Working Directory, Index, and HEAD:** The Working Directory contains actual files on disk. The Index (Staging Area) holds the proposed next commit snapshot. HEAD points to the currently checked-out commit or branch tip.
Production Application: Knowing which tree is affected by commands (`git reset --soft`, `--mixed`, `--hard`) is critical to avoiding accidental data loss in production repositories.
- Trunk-Based Development vs GitFlow:** GitFlow utilizes long-lived feature, develop, release, and hotfix branches. Trunk-Based Development relies on short-lived feature branches merged frequently into main with feature flags.
Production Application: Modern high-velocity DevOps teams use trunk-based workflows to eliminate 'merge hell', reduce cycle times, and enable continuous deployment.
- Reflog (Reference Logs) Mechanics:** Git tracks every time HEAD or any branch ref moves in local repository history via `.git/logs/`. Reflog entries persist even after hard resets, rebases, or branch deletions.
Production Application: The ultimate safety net for developers and DevOps engineers to rescue dropped branches or reversed commits after an errant `git reset --hard`.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
<code>git reflog</code> <code>git reflog [show] [ref]</code>	Display the chronological audit log of all HEAD movements and branch ref changes in the local repository. Options: [{flag: 'show', 'description': 'Display log entries for a reference (default is HEAD).'}, {flag: '--date=relative', 'description': 'Format timestamps as relative time (e.g. '2 hours ago').'}]	Inspect the reflog index (e.g. <code>HEAD@{1}</code>) to identify the SHA of the commit right before the destructive operation, then reset back to it. Pitfall: Assuming reflog is shared across remotes; reflog is strictly local to each machine.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
git rebase -i HEAD~4 git rebase -i	Initiate an interactive rebase session to reorder, squash, edit, or drop the last 4 commits on the current branch. Options: [{ 'flag': '-i', 'description': 'Open interactive text editor to specify commit actions.' }, { 'flag': '--autosquash', 'description': 'Automatically organize fixup! and squash! commits.' }]	Run on local feature branches prior to merging into protected trunk branches to maintain a clean linear commit graph. Pitfall: Rebasing shared public branches that other engineers have already pulled, causing duplicate divergent histories.
git push origin main --force-with-lease git push [options]	Safely push rewritten Git history to a remote repository only if no other collaborator has pushed commits to the remote branch in the interim. Options: [{ 'flag': '--force-with-lease', 'description': 'Atomic check: fails if remote ref has advanced past local tracking ref.' }, { 'flag': '--force', 'description': 'Blindly overwrites remote branch, risking overwriting colleague commits.' }]	Used in CI/CD pipelines and developer workstations when synchronizing rebased feature branches with GitHub/GitLab. Pitfall: Using `git push --force` instead of `--force-with-lease`, which can silently destroy work pushed by colleagues.
git bisect start && git bisect bad HEAD && git bisect good v2.1.0 git bisect [start bad good reset run]	Perform binary search across commit history to rapidly identify the exact commit that introduced a software regression or bug. Options: [{ 'flag': 'start', 'description': 'Initialize the bisect wizard state machine.' }, { 'flag': 'bad', 'description': 'Mark current or specified commit as broken/buggy.' }, { 'flag': 'good', 'description': 'Mark an older commit as known-working.' }]	DevOps and QA teams automate root-cause isolation by coupling `git bisect run` with automated test suites. Pitfall: Forgetting to run `git bisect reset` after finding the faulty commit, leaving the repository in a detached HEAD state.
git stash push -m 'wip-database-migration' --include-untracked git stash push [options] [-m ""]	Temporarily shelves modified tracked files and newly created untracked files into the stash stack to restore a clean working tree. Options: [{ 'flag': '-m ""', 'description': 'Attach a descriptive label to the stash entry.' }, { 'flag': '--include-untracked', 'description': 'Stash newly created files that have not yet been added to the index.' }, { 'flag': '--all', 'description': 'Stash untracked and ignored files as well.' }]	Execute before checking out a release branch when current experimental changes are not ready for a commit. Pitfall: Leaving stashes unnamed (`git stash`), making it difficult to identify what code was saved weeks later.

4. Security Hardening & Anti-Patterns

Security Mandates: Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

Critical Anti-Pattern: Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.